PROSIA

MANUFACTURING EXECUTION SYSTEM

Documentación Técnica
Base de Datos

Diseño del esquema relacional PostgreSQL

Estructura, propósito y decisiones de diseño

Versión 1.0 · Junio 2026 · Diego

Contenido

1. Introducción al sistema PROSIA
2. Arquitectura general del schema
   ENUMs, tablas catálogo, tablas transaccionales, convenciones de diseño
3. Grupo 1 — Catálogos base
   Personas, roles, ítems, unidades, tipos
4. Grupo 2 — Producción
   Turnos, máquinas, operaciones, órdenes, registros de tiempo
5. Grupo 3 — Inventario y bodegas
   Bodegas, ubicaciones, stock, movimientos, lotes
6. Grupo 4 — Calidad e inspección
   Parámetros, planes, inspecciones, no conformidades
7. Grupo 5 — Analítica y auditoría
   Eventos, KPIs, alertas, auditoría, sesiones
8. Relaciones clave entre módulos
9. Sugerencias técnicas

1 Introducción al sistema PROSIA

PROSIA es un **Sistema de Ejecución de Manufactura (MES)** diseñado para pequeñas y medianas empresas del sector manufacturero. Su objetivo es digitalizar y controlar los procesos productivos: desde la planificación de órdenes hasta el control de calidad y el seguimiento de inventario.

¿Qué problemas resuelve?

Problema del negocio	Módulo que lo resuelve
¿Cuánto tiempo tardó cada operario en cada operación?	Registro de tiempo (Producción)
¿Cuándo paró la máquina y por qué?	Paradas de máquina (Producción)
¿Qué materiales se consumieron en esta orden?	BOM + Lotes (Inventario)
¿Este producto pasó el control de calidad?	Inspecciones + No conformidades (Calidad)
¿Quién modificó este registro y cuándo?	Auditoría de usuario (Analítica)
¿Cuál es la eficiencia del área de ensamble este turno?	KPI Snapshots (Analítica)

> **Alcance actual**
>
> PROSIA v1 apunta a empresas de una sola planta (PYME). El esquema no incluye multiempresa ni multiplanta. El sistema admite múltiples usuarios, roles y áreas de trabajo dentro de una misma organización.

2 Arquitectura general del schema

2.1 Capas del schema

El schema está organizado en cinco capas que reflejan la jerarquía de dependencias:

```
CAPA 1 — ENUMs y funciones utilitarias (sin dependencias)
    ↓
CAPA 2 — Tablas catálogo (datos parametrizables, pocos registros)
    ↓
CAPA 3 — Tablas transaccionales de producción (el corazón del sistema)
    ↓
CAPA 4 — Tablas de inventario y calidad (dependen de producción)
    ↓
CAPA 5 — Tablas de analítica y auditoría (observan a todas las demás)
```

2.2 ENUMs — valores fijos del sistema

Un ENUM es un tipo de dato que solo admite valores predefinidos. Se usa cuando el negocio **nunca necesitará agregar nuevas opciones** sin cambiar código. PROSIA tiene 10 ENUMs:

ENUM	Valores posibles	¿Dónde se usa?
`enum_estado_item`	Activo · Inactivo · Descontinuado · Mantenimiento	tab_item
`enum_permisos_menu`	Crear · Consultar · Modificar · Eliminar	tab_permiso_menu_rol
`enum_estado_registro_tiempo`	Iniciado · Finalizado · Suspendido	tab_registro_tiempo
`enum_tipo_tercero`	Cliente · Proveedor · Mixto	tab_tercero
`enum_tipo_dato_calidad`	Numerico · Texto · Booleano	tab_parametro_calidad
`enum_estado_inspeccion`	Aprobado · Rechazado · Aprobado con Observacion · Pendiente	tab_inspeccion
`enum_estado_no_conformidad`	Abierta · En Analisis · Cerrada · Rechazada	tab_no_conformidad

`enum_severidad`	Baja · Media · Alta · Critica	tab_no_conformidad · tab_alerta
`enum_estado_alerta`	Nueva · Reconocida · Resuelta · Descartada	tab_alerta
`enum_tipo_accion_auditoria`	Login · Logout · Crear · Modificar · Eliminar · Consultar	tab_auditoria_usuario

2.3 Catálogos vs ENUMs — la decisión clave

> **Regla de oro del diseño PROSIA**
>
> Se usa ENUM cuando los valores son universales y no cambian con el negocio (ej: las acciones de auditoría siempre serán Crear/Modificar/Eliminar).
>
> Se usa tabla catálogo cuando el administrador del sistema necesita agregar o cambiar valores sin tocar código (ej: tipos de movimiento de inventario, estados de orden de producción).

2.4 Convenciones de nomenclatura

Convención	Ejemplo	Motivo
Prefijo `tab_`	`tab_item`	Distingue tablas de vistas, funciones y tipos
Prefijo `enum_`	`enum_severidad`	Identifica tipos ENUM al instante
Prefijo `id_`	`id_item`	Todo PK y FK empieza con `id_`
Prefijo `chk_`	`chk_stock`	Nombra los CHECK constraints
Prefijo `uk_`	`uk_version_bom`	Nombra los UNIQUE constraints
Snake_case	`fecha_inicio_vigencia`	Estándar PostgreSQL

2.5 Estrategia de PKs (claves primarias)

Tipo de tabla	Tipo de PK	Razón
Catálogos (pocos registros)	`DECIMAL(n,0)` manual	El administrador elige el ID con significado (1=Activo, 2=Inactivo)
Transaccionales (muchos registros)	`SERIAL` o `INT` auto	El sistema genera el ID automáticamente, sin intervención humana

Tablas de log/analítica	`DECIMAL(10,0)` manual	Consistencia (pendiente mejorar a SERIAL)

3 Catálogos base

Personas · Roles · Ítems

tab_tipo_doc

CATÁLOGO

Tipos de documento de identidad (Cédula, NIT, Pasaporte...)

Columna	Tipo	¿Qué guarda?
id_tipo_doc	DECIMAL(1,0) PK	ID manual del tipo. Máximo 9 tipos.
nombre	VARCHAR(50)	Nombre del documento. Único.
descripcion	VARCHAR(200)	Descripción larga.

tab_persona

CATÁLOGO

Datos personales de cualquier individuo en el sistema (empleado, cliente, proveedor)

Columna	Tipo	¿Qué guarda?
id_persona	DECIMAL(10,0) PK	Número de documento como PK natural
id_tipo_doc	DECIMAL(1,0) FK	Tipo de documento de identidad
nombre / apellido	VARCHAR(50)	Nombre completo
datos_contacto	JSONB	Teléfono, dirección, etc. Flexible: cada persona puede tener campos distintos
estado	BOOLEAN	TRUE = activo en el sistema

💡 **JSONB** permite almacenar datos variables sin agregar columnas. Ejemplo: `{"telefono": "300-111", "ciudad": "Cali"}`

tab_tercero

CATÁLOGO

Empresas externas: clientes, proveedores o ambos (mixto)

Columna	Tipo	¿Qué guarda?
id_tercero	DECIMAL(10,0) PK	NIT o identificador de la empresa
razon_social	VARCHAR(100)	Nombre legal
nombre_comercial	VARCHAR(100)	Nombre con el que opera comercialmente
tipo	enum_tipo_tercero	Cliente · Proveedor · Mixto
contacto	JSONB	Datos de contacto flexibles

tab_rol · tab_menu · tab_permiso_menu_rol

CATÁLOGO

Control de acceso basado en roles (RBAC)

Tabla	¿Qué hace?
tab_rol	Define los roles del sistema (Administrador, Supervisor, Operario...). Incluye `jerarquia` para saber qué rol tiene más privilegios.
tab_menu	Cada opción del menú de la aplicación. Soporta menús anidados con `id_menu_padre`.
tab_permiso_menu_rol	Tabla pivote: ¿qué puede hacer el rol X en el menú Y? Combina (id_menu, id_rol, permiso) como PK compuesta.

tab_usuario

CATÁLOGO

Cuentas de acceso al sistema PROSIA

Columna	Tipo	¿Qué guarda?
id_usuario	DECIMAL(5,0) PK	ID interno del sistema
id_persona	DECIMAL(10,0) FK	Persona real detrás de la cuenta
id_rol	DECIMAL(1,0) FK	Rol que determina sus permisos
nombre_usuario	VARCHAR(30)	Username único para login
correo	VARCHAR(100)	Email único, validado con regex en CHECK
contrasenna_hash	VARCHAR(255)	Contraseña hasheada (nunca en texto plano)

Seguridad crítica

La columna contrasenna_hash NUNCA debe almacenar la contraseña directa. El backend debe aplicar bcrypt u otro algoritmo de hashing antes de guardar.

tab_item

CATÁLOGO

Catálogo maestro de todos los materiales y productos de la empresa

Columna	Tipo	¿Qué guarda?
id_item	DECIMAL(6,0) PK	Hasta 999,999 ítems
codigo / nombre	VARCHAR UNIQUE	Código interno y nombre. Ambos únicos.
stock_actual / minimo / maximo	DECIMAL(10,2)	Control de inventario
tiempo_reabastecimiento	INTEGER	Días para reponer stock. Útil para alertas.
maneja_lote	BOOLEAN	TRUE = requiere trazabilidad por lote
id_tipo_item	DECIMAL(1,0) FK	Materia Prima / Producto en Proceso / Producto Final
estado	enum_estado_item	Activo · Inactivo · Descontinuado · Mantenimiento
especificaciones_extra	JSONB	Atributos técnicos variables por tipo de producto

4 Producción Turnos · Máquinas · Órdenes · Tiempo

tab_turno · tab_descanso · tab_horario_empleado

CATÁLOGO

Gestión de turnos y horarios personalizados por empleado

Tabla	¿Qué modela?
`tab_turno`	Plantilla del turno: nombre, hora_inicio, hora_fin. Ej: "Turno Mañana 7am–4:20pm"
`tab_descanso`	Descansos dentro del turno. Ej: Descanso 10am–10:20am, Almuerzo 1pm–2pm. Vinculados al turno por FK CASCADE.
`tab_horario_empleado`	Asignación individual. `hora_fin_real` NULL = trabaja el turno completo. Si un empleado sale a las 11:30am, `hora_fin_real = '11:30'` . Vigencia con fechas de inicio y fin.

Ejemplo real: El turno "Mañana" va de 7am a 4:20pm. Diego tiene `hora_fin_real = '11:30'`. Dos colegas tienen `hora_fin_real = '11:30'`. El resto tiene `hora_fin_real = NULL` (turno completo).

tab_area_trabajo · tab_maquina

PRODUCCIÓN

Áreas físicas de producción y maquinaria individual

Tabla	¿Qué modela?
`tab_area_trabajo`	Zona física: Corte, Lijado, Ensamble... Incluye `costo_hr_operario` y `costo_hr_maquina` para calcular costos de producción.
`tab_maquina`	Equipo individual dentro del área. Ej: "Lija de Banco 1", "Lija de Banco 2". Tiene código único, marca, modelo, año fabricación (validado con función `f_anio_actual()`).

tab_parada_maquina

PRODUCCIÓN

Registro de inactividad de máquinas para cálculo de OEE

Columna	¿Qué guarda?
id_maquina	¿Qué máquina paró?
id_usuario_reporta	¿Quién reportó la parada?
descripcion	Causa de la parada (falla eléctrica, mantenimiento, cambio de herramienta...)
fecha_inicio / fecha_fin	Intervalo de inactividad. fecha_fin NULL = máquina aún parada.
duracion_minutos	NULL al crear. Un Stored Procedure la calcula cuando se registra fecha_fin .

💡 **OEE** (Overall Equipment Effectiveness) = Disponibilidad × Rendimiento × Calidad. Esta tabla alimenta el componente de Disponibilidad.

tab_estructura_BOM

PRODUCCIÓN

Lista de Materiales — qué se necesita para fabricar cada producto

Columna	¿Qué guarda?
id_item_padre	Producto que se fabrica (ej: Silla Ref. 101)
id_item_hijo	Material que se consume (ej: Lija grano 80, Tela tapizado)
cantidad_requerida	Cuánto del hijo se necesita por unidad del padre

Constraint chk_no_autofabricable: impide que un ítem se referencie a sí mismo. UNIQUE en (padre, hijo): no se puede repetir el mismo par.

tab_version_bom

PRODUCCIÓN

Control de versiones de la lista de materiales

Cuando la receta de fabricación cambia (nuevo proveedor de tela, cambio de lija), se crea una nueva versión del BOM con fechas de vigencia. Las órdenes históricas siguen referenciando la versión antigua — la trazabilidad queda intacta. UNIQUE en (id_item, numero_version).

tab_orden_produccion · tab_detalle_orden_produccion

PRODUCCIÓN

Cabecera y detalle de las órdenes de fabricación

Tabla	Analogía	Campos clave
tab_orden_produccion	La "factura" de la orden	Fechas planificadas, cliente (tercero), estado, usuario que la creó
tab_detalle_orden_produccion	Las "líneas" de la factura	Qué ítems fabricar, cantidad planificada vs producida

tab_registro_tiempo

PRODUCCIÓN

Captura cuánto tardó cada operario en cada operación de la orden

Columna	¿Qué guarda?
id_detalle_orden_produccion	¿A qué línea de la orden corresponde?
id_ruta_operacion	¿Qué operación estaba ejecutando?
id_usuario	¿Qué operario lo hizo?
cantidad_buena / cantidad_mala	Producción buena vs rechazada en ese intervalo
tipo_registro	**'Produccion'** / **'Preparacion'** / **'Limpieza'** — captura setup, calibración y limpieza por separado
estado	enum: Iniciado · Finalizado · Suspendido

tab_historial_estado_orden

PRODUCCIÓN

Log inmutable de cada cambio de estado de una orden

Cada vez que una orden cambia de estado (Borrador → En Producción → Finalizada), se inserta un registro aquí con el estado anterior, el nuevo, el usuario que hizo el cambio y el motivo. Este registro es solo-escritura: nunca se actualiza ni elimina. Es el "rastro de papel" digital.

5 Inventario y Bodegas

tab_bodega · tab_ubicacion · tab_stock_ubicacion

INVENTARIO

Dónde físicamente está cada material

Tabla	Analogía	Detalle
tab_bodega	El edificio	Bodega Principal, Bodega Materiales...
tab_ubicacion	La estantería	Pasillo A - Estante 1 - Nivel 3. Código único dentro de la bodega.
tab_stock_ubicacion	El conteo	Cuántas unidades del ítem X hay en la ubicación Y. PK compuesta (id_item, id_ubicacion).

tab_movimiento_inventario

INVENTARIO

Cada entrada, salida o traslado de material

Columna	¿Qué guarda?
id_item	¿Qué material se movió?
cantidad	Cuánto se movió (siempre positivo)
tipo_movimiento	FK a catálogo: Entrada, Salida, Traslado...
id_ubicacion_origen / destino	NULL si no aplica (ej: una entrada no tiene origen)
motivo	Texto libre explicando el movimiento

tab_lote

INVENTARIO

Trazabilidad de grupos físicos de material

Columna	¿Qué guarda?
codigo_lote	Código impreso en la etiqueta física del lote
id_item + id_tipo_item	Qué material es y de qué tipo
cantidad_inicial	Cuánto tenía el lote al llegar. Nunca cambia.
cantidad_disponible	Cuánto queda actualmente. Se descuenta al consumir.
fecha_vencimiento	NULL si no aplica. Útil para pigmentos, adhesivos, etc.
id_tercero	Proveedor de donde vino este lote
id_movimiento_origen	El movimiento de entrada que creó este lote. SET NULL si se borra.

CHECK: `cantidad_disponible <= cantidad_inicial` — nunca puede haber más de lo que entró originalmente.

6 Calidad e Inspección

tab_parametro_calidad · tab_plan_inspeccion

CALIDAD

¿Qué se mide y cuándo se mide?

Tabla	¿Qué modela?
tab_parametro_calidad	Definición de cada cosa que se puede medir: "Longitud del asiento", "Dureza del acabado", "Aprobado/No aprobado". Tipo: Numérico, Texto o Booleano.
tab_plan_inspeccion	Para el ítem X en la operación Y, mide el parámetro Z con rango [min, max]. Es la "plantilla" de inspección.

tab_inspeccion · tab_detalle_inspeccion

CALIDAD

Registro real de cada inspección ejecutada

Tabla	Detalle
tab_inspeccion	Cabecera: quién inspeccionó, qué inspeccionó (orden de producción O movimiento de inventario), resultado general (Aprobado/Rechazado/...).
tab_detalle_inspeccion	Línea por línea: valor registrado para cada parámetro y si cumple o no. UNIQUE (inspeccion, parametro): no se repite el mismo parámetro dos veces.

El CHECK `chk_origen_inspeccion` garantiza que la inspección tiene exactamente un origen: de producción O de inventario O ninguno. No puede venir de ambos al mismo tiempo.

tab_no_conformidad

CALIDAD

Reporte formal de defectos con ciclo de vida completo

Columna	¿Qué guarda?
codigo	Código único impreso en el reporte físico (ej: NC-2026-001)
id_usuario_reporta	Quien detectó el problema
id_usuario_asignado	Quien debe resolverlo. No puede ser la misma persona que reporta.
estado_nc	enum: Abierta → En Analisis → Cerrada / Rechazada
severidad	enum: Baja · Media · Alta · Critica — qué tan grave es
accion_correctiva	Texto con la corrección aplicada. Se rellena al cerrar.
fecha_cierre	NULL mientras esté abierta. Fecha real de cierre.

7 Analítica y Auditoría

tab_evento_produccion

ANALÍTICA

Log central de todo lo que ocurre en el sistema

Cualquier evento relevante se registra aquí: inicio de orden, parada de máquina, rechazo de inspección, alerta disparada. La columna `payload` (JSONB) almacena datos adicionales específicos de cada tipo de evento. `nombre_entidad` indica a qué tabla pertenece el registro referenciado.

tab_kpi_snapshot

ANALÍTICA

Fotografías periódicas de indicadores de rendimiento

Un proceso programado (cron job) calcula KPIs cada turno o cada hora y los almacena aquí. Así el dashboard carga datos pre-calculados en lugar de hacer consultas pesadas en tiempo real. El UNIQUE compuesto (tipo_kpi, area_trabajo, turno, periodo) evita duplicados.

tab_alerta

ANALÍTICA

Alertas generadas automáticamente por el sistema

Columna	¿Qué guarda?
`estado_alerta`	Nueva → Reconocida → Resuelta / Descartada
`severidad`	Baja / Media / Alta / Critica — prioriza cuál atender primero
`id_usuario_reconoce / resuelve`	Quién tomó acción en la alerta
`fecha_reconocimiento / resolucion`	NULL mientras no se actúe. Miden tiempos de respuesta.

tab_auditoria_usuario

ANALÍTICA

¿Quién hizo qué, cuándo y desde dónde?

Columna	¿Qué guarda?
id_usuario	NULL si la sesión expiró (ON DELETE SET NULL)
tipo_accion	enum: Login · Logout · Crear · Modificar · Eliminar · Consultar
nombre_tabla / id_registro	Sobre qué tabla y qué registro específico actuó
datos_anteriores / datos_nuevos	JSONB: snapshot del registro antes y después del cambio
ip_origen / user_agent	Desde qué IP y navegador se realizó la acción

Esta tabla es de solo-escritura. Nunca debe tener UPDATE ni DELETE por ningún usuario, incluyendo el administrador. Es la evidencia forense del sistema.

tab_sesion_usuario

ANALÍTICA

Control de sesiones activas e historial de accesos

Columna	¿Qué guarda?
token_hash	Hash del token de sesión. NUNCA el token en texto claro. UNIQUE.
id_estado_sesion	FK a catálogo: Activa · Expirada · Revocada · Cerrada
fecha_ultimo_acceso	Se actualiza en cada request. Útil para detectar sesiones inactivas.
fecha_expiracion	Límite máximo de la sesión. CHECK: siempre > fecha_inicio.

8 Relaciones clave entre módulos

El hilo conductor de una orden de producción

```
tab_item ──────────────────────────────→ tab_estructura_BOM (¿con qué se fabrica?)
tab_item ──────────────────────────────→ tab_ruta_operacion (¿cómo se fabrica?)

tab_orden_produccion
    ↓ 1:N
tab_detalle_orden_produccion
    ↓ 1:N                  ↓ 1:N
tab_registro_tiempo        tab_inspeccion
                                ↓ 1:N          ↓ 1:1
                        tab_detalle_inspeccion   tab_no_conformidad
```

El hilo conductor del inventario

```
tab_bodega ──→ tab_ubicacion ──→ tab_stock_ubicacion ──→ tab_item

tab_movimiento_inventario ──→ tab_lote (trazabilidad de origen)
tab_movimiento_inventario ──→ tab_ubicacion (origen y destino)
```

El hilo conductor del usuario

```
tab_persona ──→ tab_usuario ──→ tab_rol ──→ tab_permiso_menu_rol ──→ tab_menu

tab_usuario ──→ tab_horario_empleado ──→ tab_turno ──→ tab_descanso
tab_usuario ──→ tab_sesion_usuario
tab_usuario ──→ tab_auditoria_usuario (ON DELETE SET NULL)
```

ON DELETE SET NULL vs CASCADE vs RESTRICT

CASCADE

: si borro el padre, borro los hijos (ej: borrar turno borra sus descansos — lógico).

RESTRICT

: no puedo borrar el padre si tiene hijos (ej: no puedo borrar un ítem si tiene órdenes — protege datos).

SET NULL

: si borro el padre, el hijo queda huérfano con NULL (ej: borrar usuario no borra la auditoría — conserva evidencia).

9 Sugerencias técnicas

Para consultar con instructor

Sobre esta sección

Las siguientes sugerencias son mejoras identificadas durante el diseño. No son errores — el schema funciona correctamente sin ellas. Su implementación depende del criterio del instructor y del alcance del proyecto.

S1 — PKs de tablas de alto volumen deben ser SERIAL o GENERATED IDENTITY

ALTA

Tablas afectadas: `tab_evento_produccion`, `tab_auditoria_usuario`, `tab_sesion_usuario`, `tab_kpi_snapshot`, `tab_alerta`.

Problema: Estas tablas usan `DECIMAL(10,0)` con asignación manual de ID (patrón `COALESCE + MAX`). Ese patrón funciona bien para catálogos donde un administrador inserta 5 registros al año. Pero estas tablas reciben inserciones automáticas constantes: cada acción del sistema genera un evento, cada login genera una sesión, etc. Con múltiples requests simultáneos, dos inserciones podrían calcular el mismo ID y una fallaría.

Solución: Cambiar el PK a `SERIAL` o `INT GENERATED ALWAYS AS IDENTITY`. PostgreSQL garantiza IDs únicos y consecutivos sin importar la concurrencia.

Ejemplo del cambio:

```
-- Antes
id_evento   DECIMAL(10,0) NOT NULL,

-- Después
id_evento   INT GENERATED ALWAYS AS IDENTITY,
```

S2 — Seguridad: implementar Row-Level Security (RLS) en PostgreSQL

ALTA

Actualmente el control de acceso está modelado en la aplicación (tab_rol, tab_permiso_menu_rol). Esto significa que si alguien accede directamente a la base de datos por fuera de la aplicación, puede leer o modificar cualquier tabla.

Solución: Activar Row-Level Security en PostgreSQL. Permite definir políticas a nivel de base de datos: "el usuario con rol Operario solo puede leer sus propios registros de tiempo". Incluso si se conectan directamente a la BD, las políticas se aplican.

Esta sugerencia se desarrollará en la sesión de seguridad pendiente.

S3 — Agregar índices en columnas de alta consulta (FK y filtros frecuentes)

MEDIA

PostgreSQL crea índices automáticamente solo en PRIMARY KEY y columnas UNIQUE. Las Foreign Keys y columnas usadas en filtros (`WHERE`) no se indexan automáticamente.

Columnas que se deben indexar manualmente:

- `tab_registro_tiempo.id_usuario` — consultas de "tiempo por operario"
- `tab_registro_tiempo.fecha_inicio` — reportes por rango de fechas
- `tab_movimiento_inventario.id_item` — kardex por ítem
- `tab_auditoria_usuario.id_usuario` — historial de acciones de un usuario
- `tab_auditoria_usuario.fecha_accion` — reportes de auditoría por período
- `tab_lote.codigo_lote` — búsqueda por código (ya es UNIQUE, ya tiene índice)
- `tab_sesion_usuario.token_hash` — validación de sesión en cada request (ya es UNIQUE)

Sintaxis: `CREATE INDEX idx_reg_tiempo_usuario ON tab_registro_tiempo(id_usuario);`

S4 — tipo_registro en tab_registro_tiempo debería ser un ENUM o CHECK más estricto

MEDIA

Actualmente `tipo_registro` es `VARCHAR(20) DEFAULT 'Produccion'`. No hay restricción que impida insertar valores inválidos como `'produccion'` (minúscula), `'producción'` (tilde) o `'Calibracion'` (valor no definido).

Opción A — ENUM (recomendada): Crear `enum_tipo_registro` con valores 'Produccion', 'Preparacion', 'Limpieza' y usarlo como tipo de la columna.

Opción B — CHECK constraint:

```
CONSTRAINT chk_tipo_registro CHECK (tipo_registro IN ('Produccion', 'Preparacion',
'Limpieza'))
```

S5 — Considerar particionamiento de tablas de log a futuro

BAJA

Con el tiempo, `tab_auditoria_usuario` y `tab_evento_produccion` pueden acumular millones de registros. PostgreSQL soporta particionamiento por rango de fechas: cada mes o año se convierte en una partición separada, lo que mantiene las consultas rápidas sin necesidad de archivar manualmente.

No es urgente implementarlo desde el inicio, pero es bueno diseñar sabiendo que existe esta opción.

S6 — f_anio_actual() debería documentarse como función de conveniencia

BAJA

La función `f_anio_actual()` encapsula `EXTRACT(YEAR FROM CURRENT_DATE)::integer` para usarla en el CHECK de `tab_maquina`. Es válida y funciona correctamente.

La alternativa directa es usar `EXTRACT(YEAR FROM CURRENT_DATE)` inline en el CHECK, sin crear una función separada. La función agrega una capa de abstracción que puede confundir a quien lea el schema sin leer la sección de funciones primero.

Cualquiera de las dos es válida — es decisión de estilo de código.

Resumen del estado actual del schema

El schema PROSIA v1 está estructuralmente correcto: sin FKs huérfanas, sin inconsistencias de tipos, sin ENUMs rígidos donde se necesita parametrización. Las sugerencias anteriores son mejoras incrementales de rendimiento y seguridad que se pueden implementar progresivamente conforme el sistema madure.